

Lab 01 — Containers en architectuur: Docker, Podman en de Linux-kernel

Theorie — wat een container werkelijk is, wie wat doet wanneer je `podman run` typt, en waarom "Docker" vandaag zowel een tool als een manier van werken betekent.

Doelstellingen

- Kunnen uitleggen wat een container is **zonder** de woorden "lichte virtuele machine" te gebruiken.
- De twee kernelmechanismen kunnen benoemen die containers mogelijk maken.
- De spelers uit elkaar houden: client, engine (met of zonder daemon), image, registry — bij Docker **én** bij Podman.
- Het verschil kennen tussen een **image** en een **container** — de fout die beginners het meest kost.
- Elk `docker` - of `podman` -commando kunnen lezen en de structuur ervan herkennen.

1. Een verhaal over "bij mij werkt het"

Een applicatie draait nooit op zichzelf. Een Spring Boot-API heeft een JRE nodig, en wel in een specifieke versie, plus omgevingsvariabelen, een certificaat en een tijdzone. Dat alles samen heet de **runtime-omgeving**. Twintig jaar lang installeerden we die met de hand op servers. Het gevolg laat zich raden: de laptop van de ontwikkelaar en de productieserver waren nooit helemaal gelijk, en twee applicaties op dezelfde server ruzieden om twee versies van dezelfde bibliotheek.

Het eerste antwoord was de **virtuele machine**: per applicatie een complete nagebootste computer met een eigen besturingssysteem. Dat werkt, maar de prijs is stevig: gigabytes aan schijfruimte, gereserveerd RAM en minuten opstarttijd — om *één* proces te draaien.

→ GESCHIEDENIS

In maart 2013 stelde Solomon Hykes Docker in vijf minuten voor op de PyCon-conferentie. Technisch was er niets nieuws onder de zon: *namespaces* en *cgroups* zaten al sinds 2008 in Linux, en LXC gebruikte ze al. Wat Docker wél uitvond, was de **verpakking**: een image die je bouwt, publiceert en met één commando start. In 2015 droeg Docker het image-formaat en de runtime over aan het **OCI** (*Open Container Initiative*). Sindsdien is een image een standaard die elke tool kan draaien — en uit die opening is Podman (Red Hat, 2018) ontstaan.

De container is het tweede antwoord: **isoleer het proces, niet de machine**.

2. Wat een container is

ONTHOUDEN

Een container is een **gewoon proces** op je Linux-machine. Alleen liegt de kernel tegen dat proces over wat het kan zien en hoeveel het mag verbruiken.

In een container zit geen besturingssysteem. Er wordt niets geëmuleerd. Start je een `nginx` -container, dan draait er op je host een echt `nginx` -proces, zichtbaar in `ps aux`, uitgevoerd door **dezelfde Linux-kernel** als al de rest. Wat verandert, is hoe dat proces de wereld waarneemt. Twee kernelmechanismen zorgen daarvoor.

→ LINUX

De **kernel** is het deel van het systeem dat met de hardware praat en scheidsrechter speelt tussen programma's: hij maakt processen aan en geeft ze geheugen, CPU-tijd en toegang tot bestanden en netwerk. Alles wat daarboven leeft — `bash`, `ls`, `java`, `nginx` — heet **userland**. Een programma raakt de hardware nooit rechtstreeks aan: het vraagt alles aan de kernel via **systeemaanroepen** (*system calls*: `open`, `read`, `fork` ...). Precies die grens benutten containers.

Namespaces — bepalen wat een proces ziet. De kernel geeft een proces een gedeeltelijk, privaat zicht op bepaalde resources:

Namespace	Wat hij isoleert	Zichtbaar gevolg
<code>pid</code>	De proces-ID's	In de container is je applicatie PID 1 en ziet ze niets anders
<code>net</code>	Interfaces, poorten, routes	De container heeft zijn eigen poort 8080, los van die van de host
<code>mnt</code>	De mountpunten	De container ziet zijn eigen <code>/</code>
<code>uts</code>	De hostnaam	<code>hostname</code> geeft de container-ID terug
<code>ipc</code>	Interprocescommunicatie	Geen gedeeld geheugen met de burens
<code>user</code>	De UID's/GID's	<code>root</code> in de container kan op de host een gewone gebruiker zijn — daar draait rootless Podman om

Cgroups (control groups) — begrenzen wat een proces verbruikt. Namespaces verbergen, cgroups leggen een plafond op: "deze groep processen krijgt hoogstens 512 MB RAM en 1,5 core". Zo kan een container die op hol slaat de server niet meesleuren.

Container of VM?

	Virtuele machine	Container
Isoleert	Een hele computer (gevirtualiseerde hardware)	Eén of meer processen
Kernel	Een eigen kernel	Die van de host, gedeeld
Opstart	Seconden tot minuten	Milliseconden
Typische grootte	Meerdere GB	Enkele tientallen MB
Veiligheidsgrens	Sterk (hypervisor)	Zwakker (één kernel te compromitteren)

→ WINDOWS / WSL

"De container deelt de kernel van de host" heeft een gevolg: een Linux-container draait **alleen** op een Linux-kernel. Op Windows zorgt **WSL 2** (*Windows Subsystem for Linux*) daarvoor: een heel lichte VM onder Hyper-V die in één seconde opstart, RAM deelt met Windows en een echte, door Microsoft gecompileerde Linux-kernel draait. Je Podman draait *in* die Ubuntu-distributie; je containers zijn processen van die VM, niet van Windows. Docker Desktop en Podman Desktop doen achter de schermen hetzelfde: ze maken hun eigen WSL-distributie aan.

3. Image en container

Dit is hét onderscheid waar de hele opleiding op steunt.

Een **image** is een **alleen-lezen** sjabloon: een bevroren bestandssysteem (de JRE, je JAR, de bibliotheken) plus metadata (welk commando starten, welke variabelen, welke gebruiker, welke poort). Een image voert niets uit, verbruikt geen CPU en "draait" niet. Ze is inert en **onveranderlijk**.

Een **container** is een *draaiende instantie* van een image: de image, plus een dunne schrijflaag die bij die ene instantie hoort, plus een levend proces.

→ JAVA

De beste analogie komt uit objectgeoriënteerd programmeren: de image is de **klasse**, de container het **object** (`new`). Je kunt twintig containers instantiëren van dezelfde image; ze delen dezelfde alleen-lezen inhoud en hebben elk hun eigen private toestand. En net als een object kun je een container vernietigen zonder dat de klasse er iets van merkt.

ONTHOUDEN

Alles wat je applicatie in een container schrijft, belandt in die schrijflaag — en die wordt **samen met de container vernietigd**. Dat is geen ontwerpfout maar een keuze: een container is wegwerpbaar. Persistentie komt aan bod in lab 06.

Een image bestaat uit gestapelde **lagen** (*layers*), één per bouwstap. Tien images die vertrekken van dezelfde `eclipse-temurin:21-jre` slaan die basis maar één keer op (lab 02).

4. De architectuur: wie doet wat

Hier lopen Docker en Podman uiteen — en dat is precies waarom Podman bestaat.

DOCKER	<code>docker</code> (client)	→ HTTP/socket →	<code>dockerd</code> (daemon, root)	→	<code>containerd</code>	→	<code>runc</code>
PODMAN	<code>podman</code> (jouw gebruiker)	→ fork/exec →	<code>conmon</code>	→	<code>crun</code>		(geen daemon)
	beide → pull → Registry (Docker Hub, Harbor, ECR...)						

Docker heeft een **client/server**-architectuur. Het `docker`-binary doet bijna niets zelf: het vertaalt je commando naar een HTTP-verzoek aan `dockerd`. Die permanente **daemon** draait als **root**, doet al het werk (bouwen, aanmaken, opslaan) en luistert op een Unix-socket, `/var/run/docker.sock`.

Podman heeft **geen daemon**. Elk `podman`-commando is een gewoon programma dat het werk zelf doet en dan stopt. De container blijft leven dankzij `conmon`, een piepklein toezichtsproces dat eraan gekoppeld blijft. En bovenal draait Podman standaard **rootless**: *jouw* gebruiker start de container, zonder extra privileges. De `root` die je straks in de container ziet, is een illusie van de `user`-namespace — aan hostzijde ben jij het.

PODMAN

Waarom een tweede tool? Operationele teams hadden twee klachten over Docker: **een permanente root-daemon** (één enkel faalpunt, en "wie met de socket kan praten, is root") en **een licentie** (Docker Desktop is sinds 2021 betalend voor bedrijven). Podman lost beide op: geen daemon, standaard rootless, gratis. Bovendien maakte het een beslissende keuze: de CLI is **identiek** aan die van Docker. In 95% van de gevallen volstaat `alias docker=podman`; images, Dockerfiles en registries zijn dezelfde, want dat is allemaal OCI. Je leert hier dus *Docker* — het jargon dat je op de werkvloer hoort — met Podman als motor.

De rest delen ze: de **registry**, de externe opslagplaats voor images (standaard Docker Hub; in bedrijven Harbor, Nexus, GitLab Registry, ECR, ACR), en de laagniveau-**runtime** (`runc` of `crun`),

die de kernel daadwerkelijk vraagt om de namespaces aan te maken. Die naam duikt geregeld op in foutmeldingen.

Twee praktische gevolgen die je meteen moet begrijpen:

1. **Bij Docker gebeurt het werk aan de kant van de daemon.** Een pad dat je mount met `-v /data:/data` wordt opgezocht op de schijf van de *daemon*, niet van de client. Lokaal merk je daar niets van, maar tegenover een daemon op afstand levert het eindeloos veel verrassingen op. Bij Podman zijn client en engine hetzelfde proces.
2. **Toegang tot de Docker-daemon = root-toegang op de host.** Wie naar `/var/run/docker.sock` kan schrijven, kan een geprivilegieerde container starten en de machine overnemen.

→ BEVEILIGING

Lid zijn van de groep `docker` komt neer op `sudo` zonder wachtwoord en zonder auditspoor. Rootless Podman is het structurele antwoord: een gecompromitteerde container heeft alleen *jouw* rechten.

5. Anatomie van een commando

De CLI volgt een vaste grammatica, dezelfde voor beide tools:

```
podman [object] [actie] [opties] [doel] [argumenten]
```

```
podman container run -d --name api -p 8080:8080 docker.io/library/eclipse-temurin:21-jre java -
version
#      └─object─┘ └─actie─┘ └─── opties ───┘ └─────────── image ─────────┘ └─
commando ─┘
```

De belangrijkste objecten zijn `image`, `container`, `volume`, `network` en `system` (plus `pod`, eigen aan Podman). Voor de meest gebruikte bewerkingen bestaan er **historische verkorte vormen** waarbij het object impliciet blijft — en die kom je overal tegen:

Volledige vorm	Gangbare verkorte vorm
<code>podman container run</code>	<code>podman run</code>
<code>podman container ls</code>	<code>podman ps</code>
<code>podman image ls</code>	<code>podman images</code>
<code>podman image pull</code>	<code>podman pull</code>
<code>podman container rm</code> / <code>podman image rm</code>	<code>podman rm</code> / <code>podman rmi</code>

Drie diagnosecommando's die je vanaf nu paraat moet hebben:

```
podman version  # versie van de client (en van de server, als die er is)
podman info     # toestand van de engine: rootless?, cgroups, netwerk, opslag, kernel
podman inspect  # alle metadata van een object, als JSON
```

VALKUIL

Bij Docker toont `docker version` twee blokken, *Client* en *Server*. Ontbreekt het tweede, dan draait de daemon niet of mag jij er niet mee praten — aan je commando ligt het nooit. Bij Podman zie je maar één blok, want er is geen server. De melding "Cannot connect to the Docker daemon" uit elke FAQ kan bij jou dus niet opduiken... tenzij je `podman --remote` of `podman machine` gebruikt.

6. In het bedrijf

Op een stack met Spring Boot, Angular en PostgreSQL:

- De Spring Boot-backend wordt **één image** met een JRE en de JAR. Diezelfde image, tot op de **digest** identiek, gaat naar integratie, acceptatie en productie. Daarmee is "bij mij werkt het" definitief voorbij.
- De Angular-frontend wordt *gebuid* (`ng build`), en het statische resultaat gaat in een nginx-image. Node haalt de productie niet — lab 05.
- PostgreSQL komt uit een officiële publieke image. Die schrijf je niet zelf, die configureer je.
- Deze drie images leven in een **private registry**, waar de CI ze naartoe pusht. Op de servers draaien ze onder Docker, Podman of Kubernetes: de image weet niet wie ze start, en dat is net de bedoeling.

Onthouden

- Een container is een **proces**, geïsoleerd door *namespaces* en begrensd door *cgroups*, uitgevoerd door de **kernel van de host** — geen mini-VM. Op Windows is die kernel die van WSL 2.
- Een **image** is een onveranderlijk sjabloon; een **container** is er een levende instantie van, met een wegwerpbare schrijflaag.
- Docker = client + permanente root-daemon; Podman = een programma zonder daemon, standaard rootless. Dezelfde CLI, dezelfde images, dezelfde registries (OCI).
- Containers zijn **wegwerpbaar**: alle toestand die je niet naar buiten brengt, verdwijnt zodra je ze verwijdt.
- Toegang tot de Docker-daemon = root-toegang; rootless Podman geeft alleen jouw rechten.
- De CLI volgt `podman <object> <actie>`; de korte vormen (`ps`, `run`, `images`) zijn verkorte schrijfwijzen.

Woordenschat

image: onveranderlijk sjabloon, stapel alleen-lezen lagen. — **container**: draaiende instantie van een image. — **layer** (laag): stuk bestandssysteem uit één bouwstap. — **registry**: server die images opslaat en verdeelt. — **repository**: alle versies van eenzelfde image (`postgres`). — **tag**: label van één versie (`postgres:16-alpine`). — **daemon**: permanente dienst (`dockerd`); bestaat niet bij Podman. — **rootless**: modus waarin engine en containers onder je eigen gebruiker draaien. — **namespace**: kernelmecanisme dat het zicht op een resource isoleert. — **cgroup**: verbruiksbeperking. — **runtime**: `runc` / `crun`, de component die de container werkelijk aanmaakt. — **common**: klein toezichtsproces van Podman, gekoppeld aan elke container. — **OCI**: open standaard voor images en runtimes.